

AvioNet PrintHub

Objetivo del Proyecto: Construir y desplegar un sistema de autoservicio de impresión y fotocopiado (Kiosco) 100% desatendido. El sistema debe ser capaz de recibir documentos vía WhatsApp, procesar archivos físicamente mediante USB, calcular costos dinámicos basados en la densidad de color, gestionar cobros físicos (monedas/billetes) y digitales (CoDi), y comunicarse de forma segura con el hardware de impresión local sin comprometer la red corporativa.

1. Topología y Arquitectura de Red (Modelo Híbrido)

Para mantener la integridad y las políticas de seguridad del firewall perimetral (FortiGate 60F), la arquitectura prohíbe la exposición de puertos locales hacia internet. Se adopta un modelo de extracción (Polling/Streaming) de tres capas:

1. **Capa Perimetral Cloud (El Receptor):** Un VPS económico corriendo **Ubuntu Server**. Expone un Webhook público mediante HTTPS (Nginx) para recibir de forma pasiva los eventos de la Cloud API de WhatsApp (Meta) y almacena temporalmente los documentos y metadatos en una base de datos en la nube.
2. **Capa de Negocio Local (El Motor):** Una PC Windows ejecutando la aplicación nativa en **WPF (C#)**. Realiza exclusivamente peticiones salientes seguras (HTTPS) hacia la base de datos Cloud para descargar los trabajos pendientes.
3. **Capa de Hardware Físico (Periféricos):** Red local e interfaces USB aisladas donde la aplicación WPF interactúa con la impresora (Epson WorkForce), escáneres, y el validador de efectivo (ESP32).

2. Módulo 1: Interfaz Cloud y WhatsApp API

Responsable de la interacción con el usuario móvil antes de que llegue al kiosco físico.

- **Tecnología:** Node.js/Fastify o Python desplegado en el servidor Ubuntu.
- **Flujo:**
 - El usuario envía un PDF al número de WhatsApp Business.
 - Meta dispara un Webhook (JSON) al servidor Ubuntu.
 - El servidor descarga el archivo de los servidores de Meta, le asigna un UUID y registra una tarea con estado **"Pendiente"** en la base de datos en la nube (ej. PostgreSQL).
 - Responde automáticamente al usuario: *"Documento recibido. Escanea tu código QR en el kiosco para liberar la impresión."*

3. Módulo 2: Core de Aplicación Local (WPF / .NET)

El cerebro operativo en sitio. Desarrollado en **C# con WPF** para aprovechar el renderizado nativo de interfaces XAML y el manejo de hilos del sistema operativo.

- **Interfaz de Usuario (UI):** Diseño Flat Design de alto contraste. Botones táctiles sobredimensionados (mínimo 80px). Ejecución en "Kiosk Mode" (Windows Assigned Access) bloqueando explorador y atajos de teclado.
- **Modelo de Hilos (STA y Async/Await):** Para evitar congelamientos (*UI Freezing*), todas las operaciones de red (descarga de PDFs, consultas CoDi), cálculos gráficos y monitoreo de hardware se ejecutan en el **ThreadPool** usando **Task.Run()** y **await**. La UI se actualiza únicamente devolviendo el contexto al **Dispatcher**.

4. Módulo 3: Procesamiento Gráfico, Impresión y Telemetría

Responsable de la monetización dinámica y la comunicación con el subsistema de Windows.

- **Algoritmo de Cotización de Color:**
 - WPF convierte internamente el PDF descargado en mapas de bits en memoria.
 - Un bucle paralelo evalúa la matriz de píxeles determinando si es escala de grises mediante la tolerancia cromática ($\$R \approx G \approx B$).
 - Si los píxeles a color superan el umbral (ej. 2%), la página se tasa a precio de color; en caso contrario, a precio de blanco y negro.
- **Impresión Silenciosa:** Se utiliza el Spooler de Windows y la API `PrintDocument` para enviar el archivo procesado de forma invisible al usuario, evitando los cuadros de diálogo estándar.
- **Monitoreo de Hardware (SNMP):** WPF ejecuta peticiones UDP asíncronas (Puerto 161) mediante `SharpSnmpLib` a los OIDs estándar (RFC 3805) de la impresora para evaluar:
 - Niveles de tinta/tóner (`1.3.6.1.2.1.43.11.1.1.9.1.1`).
 - Estados de error o atasco de papel (`1.3.6.1.2.1.25.3.5.1.2.1`).
 - Si hay atasco previo, el sistema suspende la recepción de dinero y muestra "Mantenimiento Requerido".
- **Escaneo:** Utilización de librerías wrapper para **TWAIN/WIA** que permiten digitalizar documentos físicos insertados en la bandeja del kiosk, convertirlos a PDF comprimido y enviarlos al correo o WhatsApp del usuario local.

5. Módulo 4: Hardware Físico y Cobro Aislado (ESP32)

Sistema de cobro en efectivo y digital de alta disponibilidad.

Pagos Digitales

- **CoDi / Tarjeta:** WPF se comunica vía API REST con un agregador bancario, genera el string del cobro, lo renderiza como código QR en la pantalla con `QRCoder` y realiza *polling* asíncrono esperando la confirmación `PAGADO` para liberar la impresora.

Efectivo (Validadores y Aislamiento Galvánico)

- **Protección Eléctrica:** El monedero (RM5) y el billeteo (NV9) operan a 12V con solenoides que generan ruido inductivo. Se utiliza un **Optoacoplador PC817** y resistencias limitadoras de 1 kΩ para aislar la etapa de potencia de 12V de la etapa de control lógico de 5V del microcontrolador. Las masas (GND) se mantienen estrictamente separadas.
- **Microcontrolador (ESP32 / ATmega):** Conectado al PC vía USB. Su firmware incluye:
 - *Watchdog Timer (WDT):* Tolerancia de 2 segundos para auto-reinicio ante bloqueos.
 - *Antirrebote y Antifraude:* Filtros en microsegundos y límite de pulsos para ignorar "pesca" de monedas.

- *Comunicación Estructurada*: Emite reportes por el puerto Serie (115200 baudios) directamente en formato **JSON** (ej. `{"evento":"moneda", "valor":10}`), facilitando el *parsing* directo en la clase `SerialPort` de WPF.

6. Módulo 5: Base de Datos Local y Auditoría (SQLite)

Repositorio local, liviano y sin servidor, diseñado para mantener el control transaccional offline.

Tabla	Función
configuracion	Parámetros del sistema (Precio B/N, Precio Color, APIs).
pagos	Log inmutable de monedas, billetes y transacciones digitales validadas.

trabajos	Registro de archivos procesados, desglose de páginas y cálculo final.
impresiones	Bitácora de envío exitoso al Spooler de impresión.
corte_caja	Consolidación diaria generada por el administrador (Total Billetes, Monedas, Digital).

7. Plan de Ejecución (Cronograma Base de 8 Semanas)

Fase	Semana	Actividad Crítica	Entregable

I	1	Configuración de infraestructura Cloud. Desarrollo de Webhook para API de WhatsApp.	VPS recibiendo JSON de Meta y guardando PDFs.
I	2	Arquitectura base WPF (MVVM) y conectividad de red segura (Polling hacia la nube).	WPF descargando trabajos pendientes en local.
II	3	Desarrollo del motor de procesamiento de imágenes. Algoritmo de color y uso del Spooler.	Cotizador automático de páginas imprimiendo silenciosamente.
II	4	Programación y soldadura del circuito de aislamiento. Firmware en ESP32 (Antirrebote).	Comunicación Serial enviando JSON de dinero insertado.
III	5	Lógica de integración serial en C#. Actualización del hilo de UI	Pantalla de cobro sumando saldo en tiempo real.

		interactivo con saldos.	
III	6	Integración de API CoDi y desarrollo del módulo de monitoreo SNMP de la impresora.	Pagos digitales activos. Bloqueos automáticos por falta de papel.
IV	7	Construcción de esquema SQLite. Integración del panel de administración oculto y cortes.	Trazabilidad y auditoría financiera completada.
IV	8	Configuración del SO (Kiosk Mode). Pruebas de estrés físico y de red. Ensamble en chasis.	Sistema en Producción.